

STUDENT MANAGEMENT SYSTEM

Mastering Embedded System Diploma

Design & Implementation Report

Author

Eng. Adel Shata

Embedded Systems
Software Engineering

GitHub: github.com/Adel-Shata

LinkedIn: [linkedin.com/in/adel-shata-a1b9a7376](https://www.linkedin.com/in/adel-shata-a1b9a7376)

Email: adel.shata.eng@gmail.com

1. Overview

A console-based Student Management System implemented in C using structures, pointers, file handling, string manipulation, and a circular queue. The system manages student records with support for adding students manually or loading them from a text file.

The software provides functionality for searching (by roll number, first name, or course), updating, deleting, and displaying student information. Each student can be associated with multiple courses from a predefined list.

Key C Concepts Demonstrated

- Structures and nested structures
- Pointers and pointer arithmetic
- Arrays and circular buffer/queue implementation
- Modular programming with header/source separation
- File I/O and string tokenization (strtok)
- Input validation (GPA range, course IDs, duplicate detection)
- Enumerations for type-safe constants
- Static helper functions for encapsulation

2. Features

Feature	Description
Add student manually	Interactive prompts for roll, name, GPA, and courses
Add students from file	Bulk load from students.txt with error handling
Search by roll number	Exact match lookup with full details display
Search by first name	Case-insensitive search, shows all matches
Search by course	Lists all students enrolled in a given course
Update student information	Modify roll, first name, last name, GPA, or courses
Add/remove student courses	Manage course enrollment per student
Delete student	Removes record and compacts circular queue
Display database statistics	Shows registered count and remaining capacity
Duplicate roll detection	Prevents duplicate roll numbers on add/update
Duplicate course detection	Prevents duplicate courses per student
Input validation	Validates GPA (0-4), course IDs, string lengths
Maximum capacity handling	Graceful handling when database is full (50 students)

3. Data Structures / Architecture

→ Core Structures

student_infos_t — Single student record

```
typedef struct {
    char    fName[50];
    char    lName[50];
    course_id_t cId[MAX_COURSES_PER_STUDENT];
    float    gpa;
    int      courseCount;
    int      roll;
} student_infos_t;
```

student_queue_t — Circular queue manager

```
typedef struct {
    unsigned int    count;
    student_infos_t *top;    // Most recently added student
    student_infos_t *tail;  // Oldest student
    student_infos_t *base;  // Start of underlying array
} student_queue_t;
```

→ Circular Storage Mechanism

The database uses a fixed-size array (`studentList[50]`) managed as a circular queue:

Mermaid — Circular Queue Diagram

```

flowchart LR
    BASE["base\nstudentList[0]"]
    S1["Student"]
    S2["Student"]
    S3["Student"]
    SN["Student"]

    BASE --> S1
    S1 --> S2
    S2 --> S3
    S3 --> SN
    SN -.wrap around.-> BASE

    TAIL["tail\noldest"] -. -> S1
    TOP["top\nnewest"] -. -> SN
  
```

- `base`: Fixed pointer to `studentList[0]`
- `tail`: Points to the oldest valid student (first inserted)
- `top`: Points to the most recently added student
- `count`: Number of valid students in the queue
- `repositionPointer()`: Handles wrap-around when incrementing/decrementing pointers

When the queue is not full, new students are added at `top + 1` (with wrap). On deletion, elements are shifted toward the gap to maintain contiguity, and `top/tail` are adjusted accordingly.

4. Functions

→ Primary Functions

Function	Purpose
<code>studentAddManually()</code>	Interactive addition of a single student with validation
<code>studentAddFromFile()</code>	Parses <code>students.txt</code> , adds valid records, handles errors per line
<code>studentFindNum()</code>	Prints total registered students and remaining slots
<code>studentFindByRoll()</code>	Searches by roll number, displays full student info
<code>studentFindByFName()</code>	Case-insensitive search by first name, lists all matches
<code>studentFindCourses()</code>	Lists all students enrolled in a given course ID
<code>studentDelete()</code>	Removes student by roll, compacts circular queue
<code>studentUpdate()</code>	Menu-driven update of roll, names, GPA, or courses

→ Helper Functions

Function	Purpose
repositionPointer()	Increments/decrements pointer with circular wrap-around
storageStatus()	Returns EMPTY, FULL, or NEITHER_FULL_NOR_EMPTY
rollStatus()	Checks if roll exists; returns ROLL_VALID or ROLL_REGISTERED
courseStatus()	Validates course ID and checks for duplicates
studentGetString()	Reads a line from stdin, strips newline
studentAddGpa()	Prompts for GPA with 0-4 validation loop
studentAddCourses()	Interactive course selection menu with duplicate checking
studentFillInfos()	Collects all fields for a new student record
studentDisplayCourses()	Prints human-readable course name for a course ID
studentDisplayInfos()	Prints formatted student record with courses

5. File Input

The system reads students.txt with one student per line. Fields are space-separated.

File Format

```
<roll> <firstName> <lastName> <gpa> <courseId1> <courseId2> ...
```

```
1001 Adel Shata 4 1 6 10 13
1002 Ahmed Ali 2.50 2 7 11 15
1003 Mohamed Hassan 4.00 3 8 12 16
```

→ Field Details

Field	Type	Constraints
Roll	Integer	Must be unique
First Name	String	Single token (no spaces)
Last Name	String	Single token (no spaces)
GPA	Float	Must be in range [0.0, 4.0]
Course IDs	Integers	1-19 corresponding to course_id_t enum

→ Error Handling

- Missing first/last name → Prompts for manual entry
- Invalid/missing GPA → Prompts for manual entry with validation
- Invalid or duplicate course ID → Skipped with warning
- No valid courses → Prompts for manual course entry

- Duplicate roll in file → Skips that line with warning
- Database full → Stops processing, reports failure

6. Program Demonstration

The following screenshots demonstrate the key functionalities of the system.

→ Main Menu

Console Output — Main Menu

```
=====
          STUDENT DATA BASE SYSTEM
=====
1. Add student manually
2. Add students from file
3. Find number of students
4. Find student by roll number
5. Find student by first name
6. Find students by course
7. Delete student
8. Update student
9. Exit
=====
Enter your choice: 9
Exiting the program...
```

The program starts with a numbered menu offering all operations.

→ Add Student Manually

Console Output — Add Student Manually

```
----- Add Student Manually -----
-----
Add student details
-----
Enter the roll number: 2001
Enter the first name of the student: John
Enter the last name of the student: Doe
Enter student's GPA: 3.5

1: MATH1
2: MATH2
3: MATH3
4: MATH4
5: MATH5
6: OPERATING_SYSTEMS
7: CONTROL1
8: CONTROL2
9: CONTROL3
10: ELECTRONICS1
11: ELECTRONICS2
```

```

12: CIRCUIT1
13: CIRCUIT2
14: CIRCUIT3
15: DEUTSCH
16: ENGLISH
Enter course's ID: 1
The course with ID= 1 has been added properly
Enter Y if you want to add another course, and other char if no:
n
Student has been added properly.
Registered student = 17
Remaining places   = 33

Press Enter to continue...

```

Interactive prompts guide the user through entering roll, names, GPA (validated 0-4), and courses (selected by ID from a displayed list).

→ Load Students from File

Console Output — File Loading

```

----- Add Students From File -----
Student with roll: 1001 has been added successfully
Student with roll: 1002 has been added successfully
Student with roll: 1003 has been added successfully
Student with roll: 1004 has been added successfully
Student with roll: 1005 has been added successfully
Student with roll: 1006 has been added successfully
Student with roll: 1007 has been added successfully
Student with roll: 1008 has been added successfully
Student with roll: 1009 has been added successfully
Student with roll: 1010 has been added successfully
We can not add the written student in the file with roll = 1002
because the written roll is registered before with another student
The course in the file with id = 99 for the student with roll: 1011
is invalid or already registered before
Student with roll: 1011 has been added successfully
The course in the file with id = 1 for the student with roll: 1012
is invalid or already registered before
Student with roll: 1012 has been added successfully
You have reached maximum course number can be registered to the student with roll: 1013
Student with roll: 1013 has been added successfully
There are no courses in the file for the student with roll: 1014
It must be at least one course assigned to the student
so you have to enter it manually
1: MATH1
2: MATH2
3: MATH3
4: MATH4
5: MATH5
6: OPRERATING_SYSTEMS
7: CONTROL1
8: CONTROL2
9: CONTROL3
10: ELECTRONICS1

```

```

11: ELECTRONICS2
12: CIRCUIT1
13: CIRCUIT2
14: CIRCUIT3
15: DEUTSCH
16: ENGLISH
Enter course's ID: 1
The course with ID= 1 has been added properly
Enter Y if you want to add another course, and other char if no:
n
Student with roll: 1014 has been added successfully
Invalid GPA
Please insert student's GPA manually: 3.5
Student with roll: 1015 has been added successfully
Invalid GPA
Please insert student's GPA manually: 3.0
Student with roll: 1016 has been added successfully

Students from file have been processed.
Registered student = 16
Remaining places   = 34

Press Enter to continue...

```

Loads students from students.txt. Demonstrates handling of duplicate roll, invalid course, duplicate course, too many courses, missing courses, and invalid GPA values — all reported inline without stopping the import.

→ Search by Roll Number

Console Output — Search by Roll

```

----- Find Student By Roll -----
Enter the roll number:
1001
[Student Infos]
Student Roll      : 1001
Student first Name: Adel
student Last Name: Shata
[Registered Courses]
1. →ID: 1        →MATH1
2. →ID: 6        →OPERATING SYSTEMS
3. →ID: 10       →ELECTRONICS1
4. →ID: 13       →CIRCUIT2

Press Enter to continue...

```

Exact match lookup displays the student's full record including all enrolled courses with readable names.

→ Search by First Name

Console Output — Search by Name

```

----- Find Student By First Name -----
Enter student's Name:
Adel
[Students Infos]
1st Student:
Student Roll      : 1001
Student first Name: Adel
student Last Name: Shata
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 6      →OPERATING SYSTEMS
3. →ID: 10     →ELECTRONICS1
4. →ID: 13     →CIRCUIT2

Press Enter to continue...
=====
                STUDENT DATA BASE SYSTEM
=====
1. Add student manually
2. Add students from file
3. Find number of students
4. Find student by roll number
5. Find student by first name
6. Find students by course
7. Delete student
8. Update student
9. Exit
=====
Enter your choice: 6

----- Find Students By Course -----
1: MATH1
2: MATH2
3: MATH3
4: MATH4
5: MATH5
6: OPERATING_SYSTEMS
7: CONTROL1
8: CONTROL2
9: CONTROL3
10: ELECTRONICS1
11: ELECTRONICS2
12: CIRCUIT1
13: CIRCUIT2
14: CIRCUIT3
15: DEUTSCH
16: ENGLISH
Enter the course ID needed to find the students registered in:
[Course Info]
Course Name: MATH1

```

```
1st Student:
Student Roll      : 1001
Student first Name: Adel
student Last Name: Shata
[Registered Courses]
1. →ID: 1        →MATH1
2. →ID: 6        →OPERATING SYSTEMS
3. →ID: 10       →ELECTRONICS1
4. →ID: 13       →CIRCUIT2
2st Student:
Student Roll      : 1006
Student first Name: Mahmoud
student Last Name: Samir
[Registered Courses]
1. →ID: 1        →MATH1
2. →ID: 2        →MATH2
3. →ID: 3        →MATH3
4. →ID: 4        →MATH4
5. →ID: 5        →MATH5
3st Student:
Student Roll      : 1008
Student first Name: Karim
student Last Name: Tarek
[Registered Courses]
1. →ID: 1        →MATH1
4st Student:
Student Roll      : 1011
Student first Name: Invalid
student Last Name: Course
[Registered Courses]
1. →ID: 1        →MATH1
2. →ID: 5        →MATH5
5st Student:
Student Roll      : 1012
Student first Name: Duplicate
student Last Name: Course
[Registered Courses]
1. →ID: 1        →MATH1
2. →ID: 2        →MATH2
6st Student:
Student Roll      : 1013
Student first Name: TooMany
student Last Name: Courses
[Registered Courses]
1. →ID: 1        →MATH1
2. →ID: 2        →MATH2
3. →ID: 3        →MATH3
4. →ID: 4        →MATH4
5. →ID: 5        →MATH5
6. →ID: 6        →OPERATING SYSTEMS
7. →ID: 7        →CONTROL1
8. →ID: 8        →CONTROL2
9. →ID: 9        →CONTROL3
10. →ID: 10      →ELECTRONICS1
7st Student:
Student Roll      : 1014
Student first Name: No
```

```

student Last Name: Courses
[Registered Courses]
1. →ID: 1      →MATH1
8st Student:
Student Roll    : 1015
Student first Name: High
student Last Name: GPA
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 2      →MATH2
9st Student:
Student Roll    : 2001
Student first Name: John
student Last Name: Doe
[Registered Courses]
1. →ID: 1      →MATH1

Press Enter to continue...

```

Case-insensitive search finds all students matching the given first name.

→ Search by Course

Console Output — Search by Course

```

----- Find Students By Course -----
1: MATH1
2: MATH2
3: MATH3
4: MATH4
5: MATH5
6: OPRERATING_SYSTEMS
7: CONTROL1
8: CONTROL2
9: CONTROL3
10: ELECTRONICS1
11: ELECTRONICS2
12: CIRCUIT1
13: CIRCUIT2
14: CIRCUIT3
15: DEUTSCH
16: ENGLISH
Enter the course ID need1
ed to find the students registered in:
[Course Info]
Course Name: MATH1
1st Student:
Student Roll    : 1001
Student first Name: Adel
student Last Name: Shata
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 6      →OPERATING SYSTEMS
3. →ID: 10     →ELECTRONICS1
4. →ID: 13     →CIRCUIT2
2st Student:

```

```
Student Roll      : 1006
Student first Name: Mahmoud
student Last Name: Samir
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 2      →MATH2
3. →ID: 3      →MATH3
4. →ID: 4      →MATH4
5. →ID: 5      →MATH5
3st Student:
Student Roll      : 1008
Student first Name: Karim
student Last Name: Tarek
[Registered Courses]
1. →ID: 1      →MATH1
4st Student:
Student Roll      : 1011
Student first Name: Invalid
student Last Name: Course
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 5      →MATH5
5st Student:
Student Roll      : 1012
Student first Name: Duplicate
student Last Name: Course
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 2      →MATH2
6st Student:
Student Roll      : 1013
Student first Name: TooMany
student Last Name: Courses
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 2      →MATH2
3. →ID: 3      →MATH3
4. →ID: 4      →MATH4
5. →ID: 5      →MATH5
6. →ID: 6      →OPERATING SYSTEMS
7. →ID: 7      →CONTROL1
8. →ID: 8      →CONTROL2
9. →ID: 9      →CONTROL3
10. →ID: 10    →ELECTRONICS1
7st Student:
Student Roll      : 1014
Student first Name: No
student Last Name: Courses
[Registered Courses]
1. →ID: 1      →MATH1
8st Student:
Student Roll      : 1015
Student first Name: High
student Last Name: GPA
[Registered Courses]
1. →ID: 1      →MATH1
2. →ID: 2      →MATH2
```

```

9st Student:
Student Roll      : 2001
Student first Name: John
student Last Name: Doe
[Registered Courses]
1. →ID: 1        →MATH1

Press Enter to continue...

```

Lists all students enrolled in the selected course (e.g., MATH1 = ID 1).

→ Update Student

Console Output — Update Student

```

----- Update Student -----
Enter student' roll number, to be updated:
1001
Please enter your choice:-
1. Update student's roll
2. Update student's first name
3. Update student's last name
4. Update student's GPA
5. Update student's courses
Enter your choice:
2
Enter the new first name
AdelUpdated
Enter Y if you want to perform another update, and other char if no:
n

Press Enter to continue...

```

Menu-driven update allows changing roll (with duplicate check), first name, last name, GPA, or courses (add/remove). Multiple updates can be chained in one session.

→ Delete Student

Console Output — Delete Student

```

----- Delete Student -----
Enter roll number for student to be deleted:
1001
Student has been deleted properly.
Registered student = 16
Remaining places   = 34

Press Enter to continue...

```

Removes a student by roll number. The circular queue is compacted and statistics are updated.

→ Input Validation Examples

Console Output — Input Validation

```

----- Add Student Manually -----
-----
Add student details
-----
Enter the roll number:
2002
Enter the first name of the student:
Test
Enter the last name of the student:
User
Enter student's GPA:
5.0
Invalid GPA, Please insert it from 0 → 4
Enter student's GPA:
3.0
1: MATH1
2: MATH2
3: MATH3
4: MATH4
5: MATH5
6: OPRERATING_SYSTEMS
7: CONTROL1
8: CONTROL2
9: CONTROL3
10: ELECTRONICS1
11: ELECTRONICS2
12: CIRCUIT1
13: CIRCUIT2
14: CIRCUIT3
15: DEUTSCH
16: ENGLISH
Enter course's ID: 1
The course with ID= 1 has been added properly
Enter Y if you want to add another course, and other char if no:
n
Student has been added properly.
Registered student = 17
Remaining places   = 33

Press Enter to continue...

```

Shows validation for GPA (must be 0-4), course IDs (must be 1-16), and duplicate roll detection.

→ Duplicate Roll Detection

Console Output — Duplicate Roll

```

----- Add Student Manually -----
-----
Add student details
-----
Enter the roll number:
2002
Enter the first name of the student:
Test
Enter the last name of the student:
User
Enter student's GPA:
5.0
Invalid GPA, Please insert it from 0 → 4
Enter student's GPA:
3.0
1: MATH1
2: MATH2
3: MATH3
4: MATH4
5: MATH5
6: OPRERATING_SYSTEMS
7: CONTROL1
8: CONTROL2
9: CONTROL3
10: ELECTRONICS1
11: ELECTRONICS2
12: CIRCUIT1
13: CIRCUIT2
14: CIRCUIT3
15: DEUTSCH
16: ENGLISH
Enter course's ID: 1
The course with ID= 1 has been added properly
Enter Y if you want to add another course, and other char if no:
n
Student has been added properly.
Registered student = 17
Remaining places   = 33

Press Enter to continue...

=====
          STUDENT DATA BASE SYSTEM
=====
1. Add student manually
2. Add students from file
3. Find number of students
4. Find student by roll number
5. Find student by first name
6. Find students by course
7. Delete student
8. Update student

```

```
9. Exit
=====
Enter your choice: 9

Exiting the program...
```

Attempting to add a student with an already-registered roll number is rejected with a clear error message.

→ Database Statistics

Console Output — Database Statistics

```
----- Students Number -----
Registered student = 16
Remaining places   = 34

Press Enter to continue...
```

Shows current registered student count and remaining capacity at any time.

7. Source Code

The complete source code is available in the GitHub repository:

Repository: github.com/Adel-Shata/Mastering_Embedded_System

Source Files

File	Description
Libraries.h	Common includes (stdio, stdlib, string) and print macro
Student_Data_Base.h	Data type definitions, enumerations, and function declarations
Student_Data_Base.c	Implementation of all student management functions (820 lines)
Studentt_Management_System.c	Main menu and program entry point
students.txt	Test data file with 16 students and edge cases

Refer to the repository for the complete implementation details, including the circular queue mechanism, file parsing logic, and input validation routines.